

Attention 과 Transformer

1. 어텐션(Attention)이란?

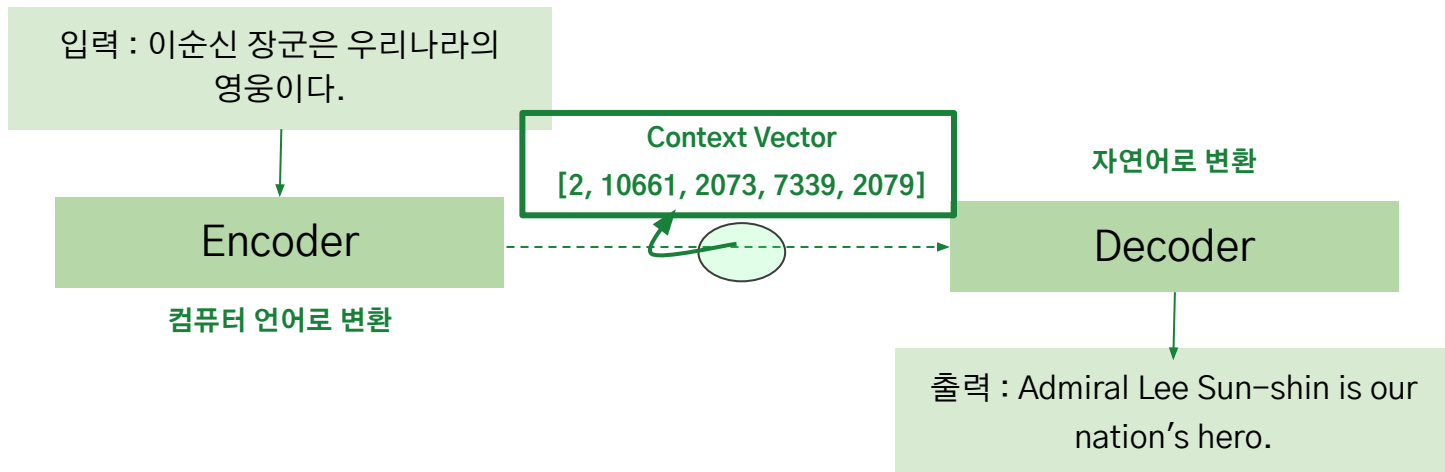
2. 트랜스포머(Transformer)

3. 허깅페이스 트랜스포머즈(transformers)

0. 지난 시간 복습

1.2 인코더(Encoder)와 디코더(Decoder)

- Seq2Seq (Sequence to Sequence) 구조
 - 인코더 (Encoder)와 디코더 (Decoder) 를 결합한 구조
 - 기계번역 기준, 입력 언어를 고정된 길이의 벡터로 변환한 후에, (고정된 길이의 벡터를) 타겟 언어로 번역하는 구조

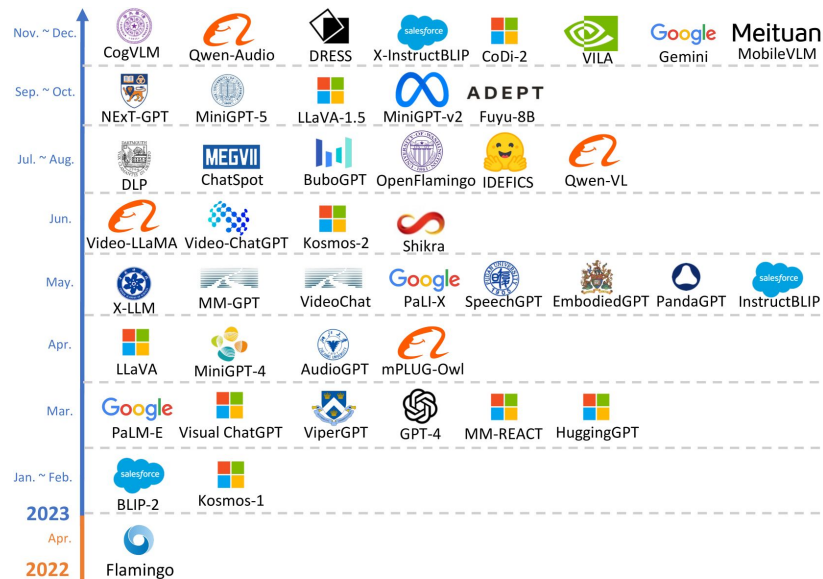


0. 지난 시간 복습

1.3 언어모델(Language Model)

- 언어모델이란?

- 언어 모델은 주어진 문맥에서 다음 단어를 예측하거나 문장의 구조와 의미를 이해하여 텍스트를 생성하는 데 사용된다.
- 언어 모델은 텍스트 데이터를 학습하여, 단어들의 확률 분포를 예측합니다.



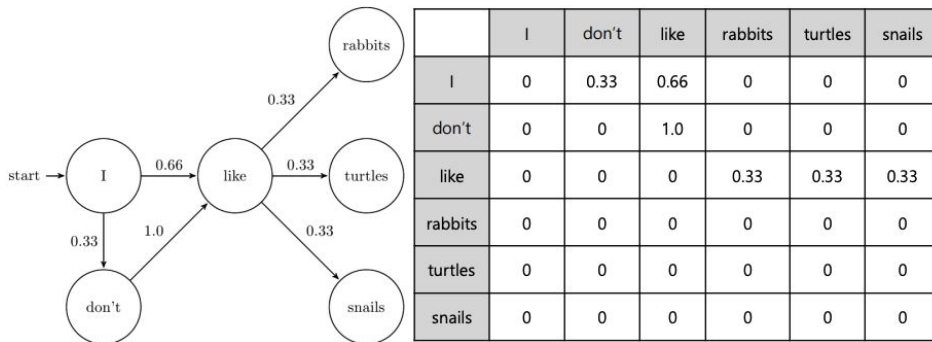
0. 지난 시간 복습

1.3 언어모델(Language Model)

- 마코브 체인(Markov Chain) - 초기의 언어모델

- 현재 상태가 주어지면 미래의 상태가 과거 상태와 상관없이 현재 상태에 의해서만 결정된다는 개념을 기반으로 한다.
- 이 모델은 통계적 예측과 의사결정 문제에서 널리 사용되며, 특히 순차적 데이터를 처리할 때 유용합니다.

$$P(X_{n+1} = s_{n+1} | X_n = s_n, X_{n-1} = s_{n-1}, \dots, X_0 = s_0) = P(X_{n+1} = s_{n+1} | X_n = s_n)$$



"I like rabbits"

"I like turtles"

"I don't like snails"



$$0.66 * 0.33 = 0.22$$

$$0.66 * 0.33 = 0.22$$

$$0.33 * 1.0 * 0.33 = 0.11$$

0. 지난 시간 복습

3.2 워드투벡터(Word2Vec)

- CBOW(Continuous Bag of Words)

- CBOW는 주어진 문맥(Context)으로부터 타깃 단어를 예측하는 방식의 워드 임베딩(Word Embedding) 기법
- 입력(Input): 문맥 단어들 (Context Words)
- 출력(Output): 중심 단어(Target Word)

중심 단어

↓

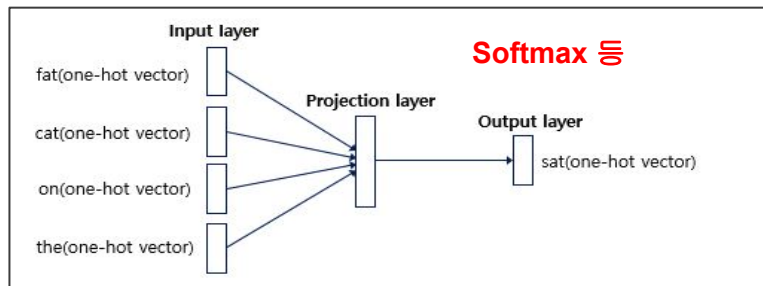
The fat cat sat on the mat

주변 단어

↓

The fat cat sat on the mat

중심 단어	주변 단어
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0, 1]



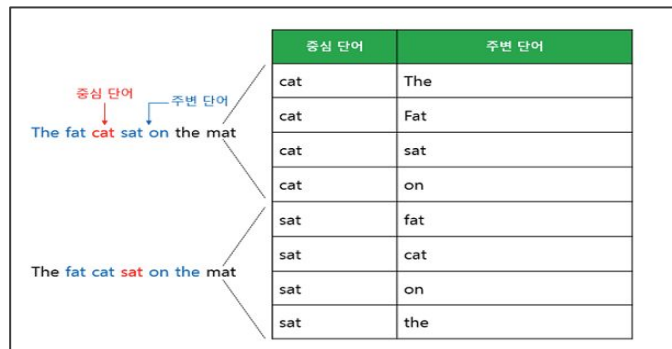
여러 단어들의 평균을 타깃(목표)로 예측하기
때문에 연산이 간단하고 빠르다.

0. 지난 시간 복습

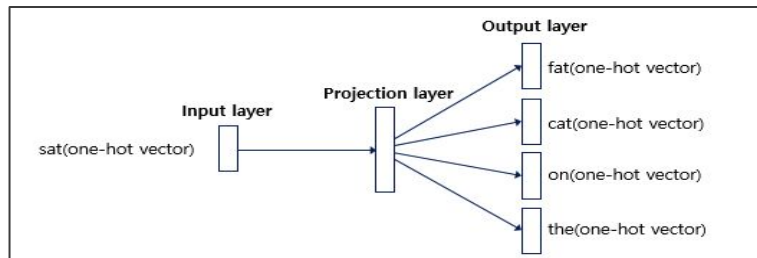
3.2 워드투벡터(Word2Vec)

- Skip-gram

- **Skip-gram**은 주어진 타겟 단어로부터 문맥(주변) 단어들을 예측하는 방식의 워드 임베딩 (Word Embedding) 기법
- **입력(Input):** 중심 단어(Target Word)
- **출력(Output):** 문맥 단어들 (Context Words)



Softmax 또는 Negative Sampling



희귀 단어에 대해서 더 나은 학습 성능을 보인다.

0. 지난 시간 복습

3.2 워드투벡터(Word2Vec)

- Word2Vec 예시

1. 도쿄는 일본의 수도, 그렇다면 서울은 어느나라의 수도일까?

일본-도쿄+서울

QUERY

+일본/Noun +서울/Noun -도쿄/Noun

RESULT

한국/Noun

2. 야구하면 박찬호, 축구하면?

박찬호-야구+축구

QUERY

+박찬호/Noun +축구/Noun -야구/Noun

RESULT

호나우두/Noun

0. 지난 시간 복습

3.4 토큰나이저(Tokenizer)

토큰나이저 역할

- 문장 텍스트를 토큰으로 나누는 도구
- 토큰의 기준은 단어, 구, 문자열 등이 될 수 있다.
- 토큰을 나누는 기준에 따라서 여러 종류의 토큰나이저가 있다.

0. 지난 시간 복습

3.4 토큰나이저(Tokenizer)

토큰나이저 동작과정

- 단계 1: 텍스트를 단어 단위로 분리 (간단하게 공백 기준)
- 단계 2: 토큰별로 고유한 인덱스를 부여하여 어휘 사전(vocab) 구축
- 단계 3: 새로운 텍스트가 들어오면 vocab 기준으로 토큰을 인덱스로 변환

“아버지가 방에 들어가신다.”



["아버지가", "방에", "들어가신다", "."]



[32, 5, 135, 3]

0. 지난 시간 복습

3.4 토큰나이저(Tokenizer)

형태소 분석기 - OKt

세종 말뭉치(corpus) 기반으로 **정해진 규칙대로** 토큰나이징한다.
품사를 같이 출력하기에 명사, 동사만 출력가능함.

```
1 from konlpy.tag import Okt
2
3 okt = Okt()
4
5 sentence = "나는 한국어를 공부합니다."
6
7 tokens = okt.morphs(sentence, stem=True)
8
9 print(tokens)
```

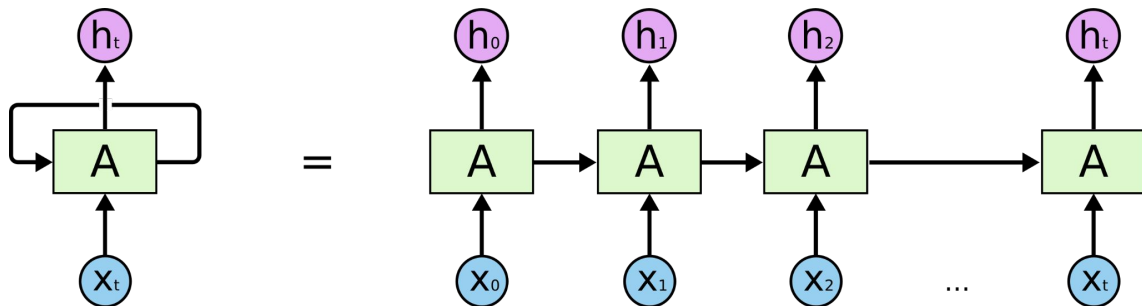
['나', '는', '한국어', '를', '공부', '하다', '.']

0. 지난 시간 복습

4.1 Recurrent Neural Network

- RNN 기본구조

- 재귀적으로 계산되는 원리의 모델 (이전 step의 정보를 이용해 다음 step 계산)
- 이전 스텝($t-1$)의 활성화 함수를 통해 나온 결과값을 출력층 뿐 아니라, 다음 스텝(t)으로 전달하여 활용

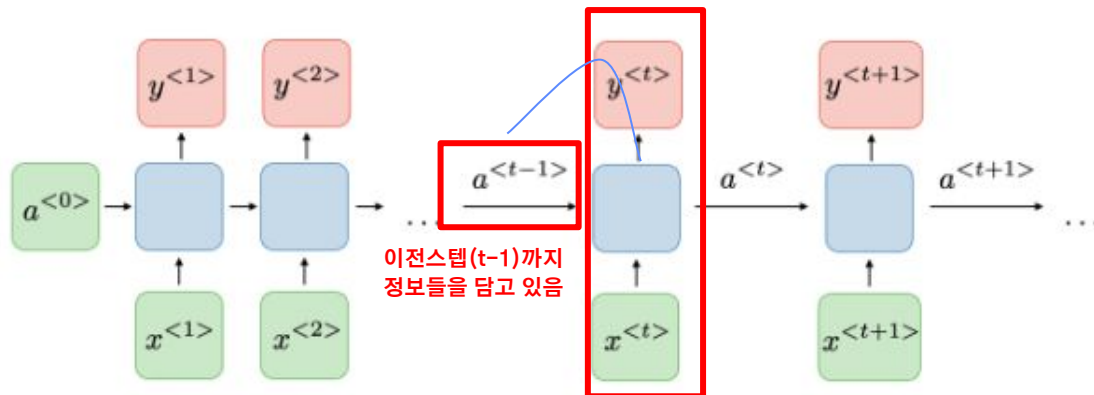


0. 지난 시간 복습

4.1 Recurrent Neural Network

- RNN 기본구조

- 재귀적으로 계산되는 원리의 모델 (이전 step의 정보를 이용해 다음 step 계산)
- 이전 스텝($t-1$)의 활성화 함수를 통해 나온 결과값을 출력층 뿐 아니라, 다음 스텝(t)으로 전달하여 활용

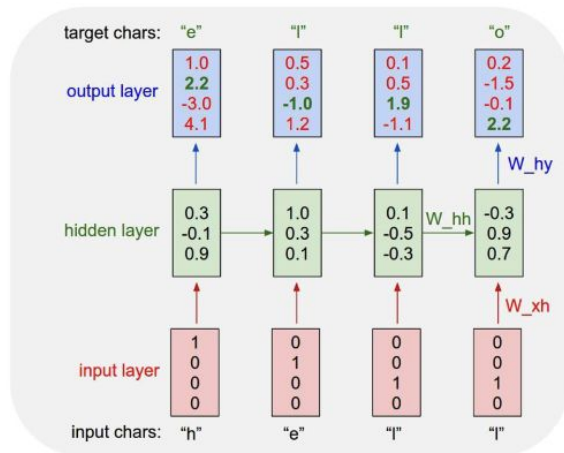


0. 지난 시간 복습

4.3 RNN 의 구조

- RNN 입출력 및 수식

- RNN은 시퀀셜 데이터 (시계열 데이터) 처리를 위해 설계된 모델이므로, 각각의 시간 단계 t 에서의 계산은 이전 시간 단계 $t-1$ 에서의 상태를 활용해 입력을 처리하는 방식이다.
- RNN에서 중요한 부분은 이전 상태 h_{t-1} 과 현재 입력 x_t 를 합쳐서 새로운 상태 h_t 를 계산하는 것.

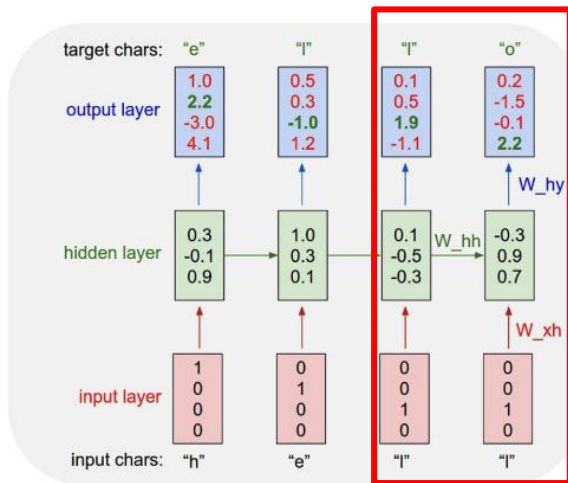


0. 지난 시간 복습

4.3 RNN 의 구조

• RNN 입출력 및 수식

1. RNN의 hidden state :



$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

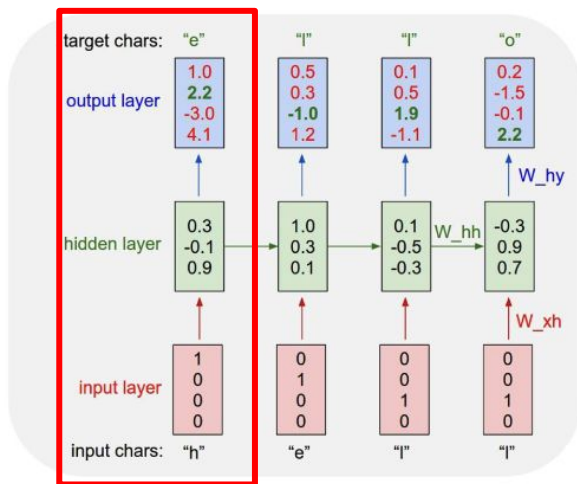
- h_t 는 시간 t 에서의 hidden state.
- h_{t-1} 는 이전 시간의 hidden state.
- x_t 는 시간 t 에서의 입력.
- W_{hh} 는 hidden state에서 hidden state로 가는 가중치 행렬.
- W_{xh} 는 입력에서 hidden state로 가는 가중치 행렬.
- b_h 는 bias term.
- f 는 활성화 함수 (보통 $\tanh$ 또는 $ReLU$).

0. 지난 시간 복습

4.3 RNN 의 구조

- RNN 입출력 및 수식

2. RNN의 출력(output) :



$$y_t = g(W_{hy}h_t + b_y)$$

- y_t 는 출력값.
- W_{hy} 는 hidden state에서 출력으로 가는 가중치 행렬.
- g 는 출력 활성화 함수 (보통 softmax나 tanh).

0. 지난 시간 복습

5.1 RNN의 한계

• 그레디언트 배니싱(Gradient Vanishing) 문제

- 연쇄법칙(chain rule)에 의해 속미분된 계수 값이 계속 곱해진다.
- 수식 예시 : $f'(3 * g'(3 * h'(3 * k'(x)))) \dots$

- Vanilla RNN에서 t 번째 시점의 hidden state는 다음과 같이 계산됩니다:

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_x x^{(t)} + b)$$

- Chain Rule을 사용하여, RNN의 역전파 과정에서 각 시점의 그레디언트를 계산합니다.

네 번째 시점의 손실 $J^{(4)}$ 에 대한 첫 번째 시점의 hidden state $h^{(1)}$ 의 Gradient:

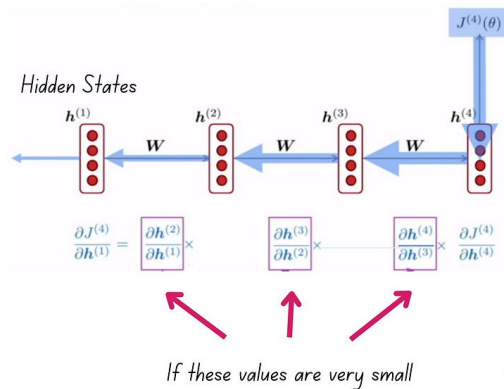
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

이 과정을 일반화하면, 첫 번째 시점의 손실 $J^{(t)}$ 에 대한 $h^{(1)}$ 의 Gradient는 다음과 같이 표현됩니다:

$$\frac{\partial J^{(t)}}{\partial h^{(1)}} = \frac{\partial J^{(t)}}{\partial h^{(t)}} \prod_{i=1}^{t-1} \frac{\partial h^{(i+1)}}{\partial h^{(i)}}$$

$J^{(t)}$ 를 $h^{(t)}$ 로 미분했을때의 속미분 계수값들의 곱

Back Propagation through Time (BPTT)



Sagar Patil

0. 지난 시간 복습

5.1 RNN의 한계

- **그레디언트 배니싱(Gradient Vanishing) 문제**

- 연쇄법칙(chain rule)에 의해 속미분된 계수 값이 계속 곱해진다.
- 수식 예시 : $f'(0.3 * g'(0.3 * h'(0.3 * k'(x)))) \dots$

- **Vanilla RNN**에서 t 번째 시점의 hidden state는 다음과 같이 계산됩니다:

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_x x^{(t)} + b)$$

- **Chain Rule**을 사용하여, RNN의 역전파 과정에서 각 시점의 그레디언트를 계산합니다.

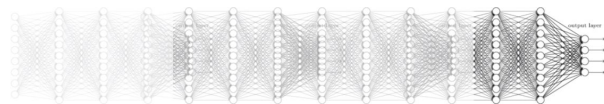
네 번째 시점의 손실 $J^{(4)}$ 에 대한 첫 번째 시점의 hidden state $h^{(1)}$ 의 Gradient:

$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

이 과정을 일반화하면, 첫 번째 시점의 손실 $J^{(t)}$ 에 대한 $h^{(1)}$ 의 Gradient는 다음과 같이 표현됩니다:

$$\frac{\partial J^{(t)}}{\partial h^{(1)}} = \frac{\partial J^{(t)}}{\partial h^{(t)}} \prod_{i < t} \frac{\partial h^{(i+1)}}{\partial h^{(i)}}$$

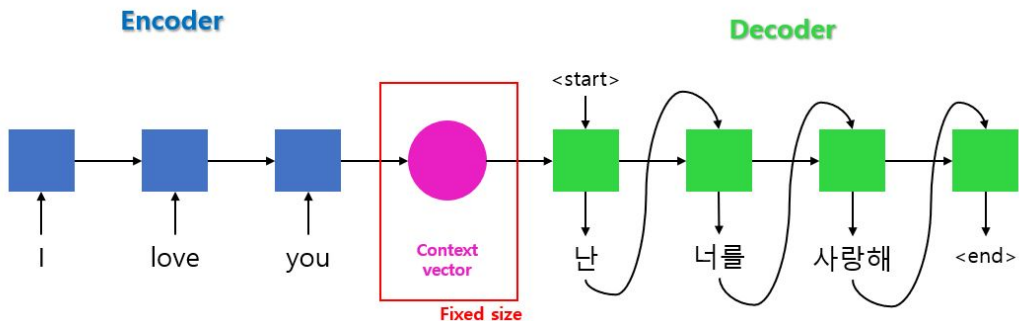
Vanishing gradient (NN winter2: 1986-2006)



0. 지난 시간 복습

5.1 RNN의 한계

- 병목(Bottle neck) 현상
 - 한정된 크기의 Context Vector 를 통해 Encoder 의 내용을 Decoder 에 전달함.



0. 지난 시간 복습

5.1 RNN의 한계

- 병목(Bottle neck) 현상

- Encoder 측 : ㅎㅎ 100가지 이야기를 해놓고 1가지로 요약해 드릴게요.

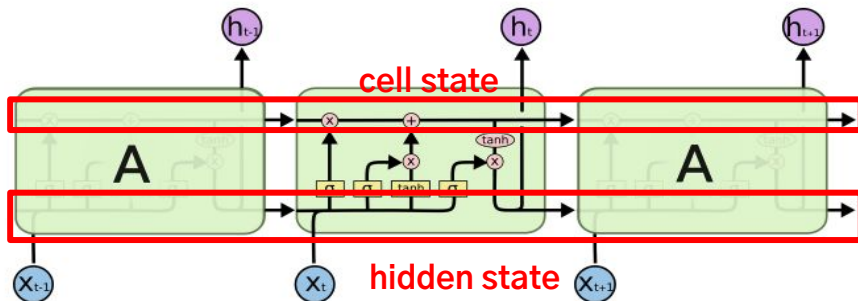
- Decoder 측 : 장난하냐.. 이 1장 보고 100장 내용을 어케알아!!

0. 지난 시간 복습

5.2 LSTM

- Long Short Term Memory

- Vanilla RNN 의 단점인 장기 의존성 문제를 보완하여 나온 RNN 모델
- LSTM 은 **cell state** 에 핵심 정보들을 모두 담아두고, 각 타임 스텝 (time step)마다 필요한 정보만을 hidden state 에 업데이트하여 사용되도록 학습
- 오차역전파 (backpropagation) 진행시 RNN 은 **그레디언트 소실 문제 (Gradient Vanishing Problem)**가 있었지만, LSTM 은 이 문제를 **cell state** 와 **게이트 (gate)**라는 개념을 통해 **그레디언트 소실 문제를 개선함**

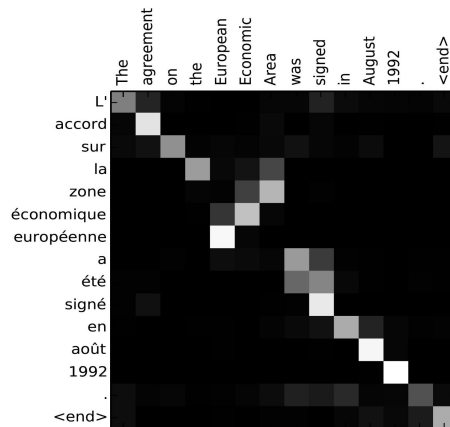


1.어텐션(Attention)이란?

1. 어텐션(Attention)이란?

1.1 어텐션(Attention)

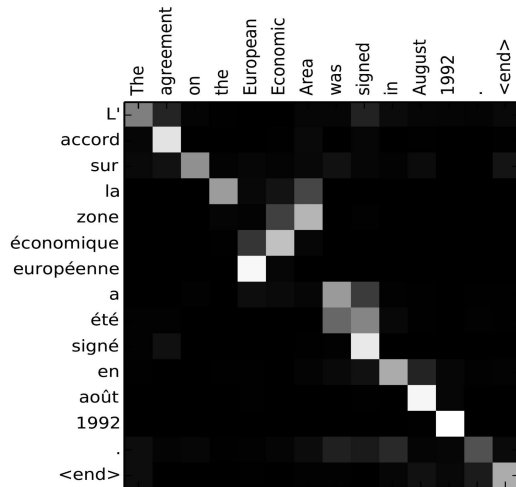
- 사람이 사물을 보는 방식
- 사람은 사물을 볼 때, 시야 내의 모든 부분을 동일하게 보지않고 **일부에 집중 (Attention)**하게 된다.



1. 어텐션(Attention)이란?

1.1 어텐션(Attention)

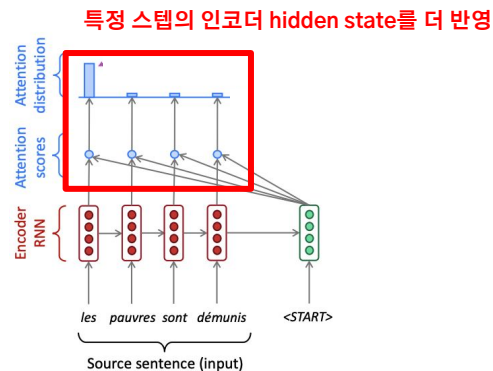
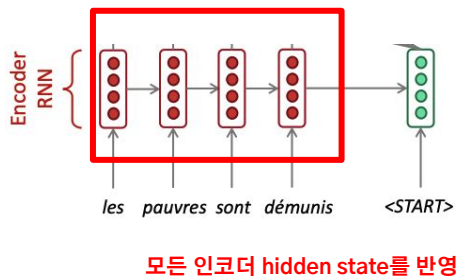
- 사람이 사물을 보는 방식
 - 사람은 사물을 볼 때, 시야 내의 모든 부분을 동일하게 보지 않고 **일부에 집중 (Attention)**하게 된다.
 - 적용 : 단어들 사이에서도 그 스텝(t)에서 어디에 집중할지 모델에 알려주면 더 문맥을 잘 이해하지 않을까?!



1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

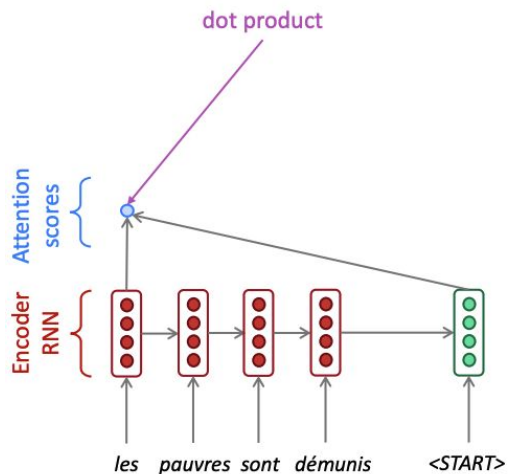
- Seq2Seq RNN 구조 위에 Attention 구조를 추가한 형태로,
- 디코더의 특정 예측 시점에서 디코더의 hidden state 가 인코더의 어떤 스텝의 hidden state 에 유사한 지 계산하여, **디코더가 어디에 집중 (Attention)할 지 반영하는 형태로 학습**



1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

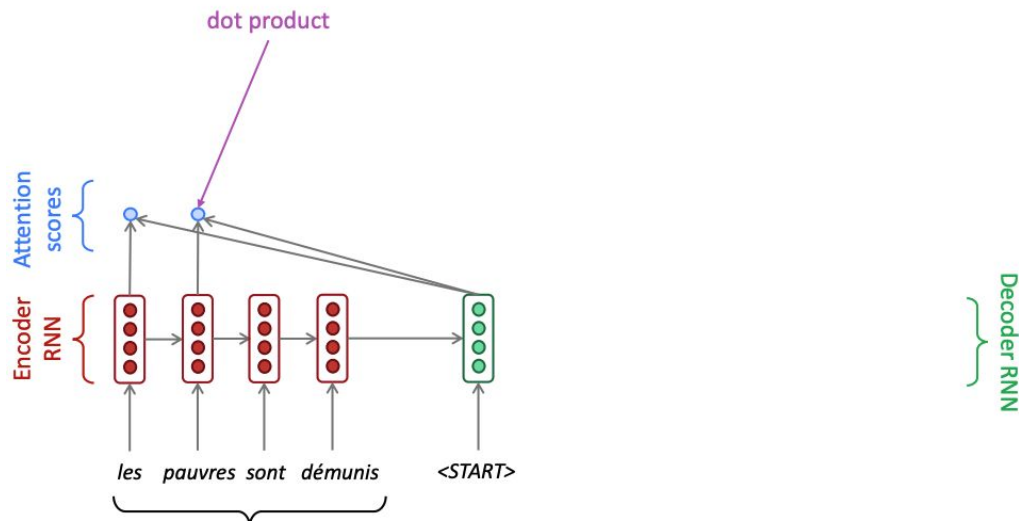
- 기계번역 예시
- 디코더 step 1 에서의 어텐션 with 인코더 1



1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

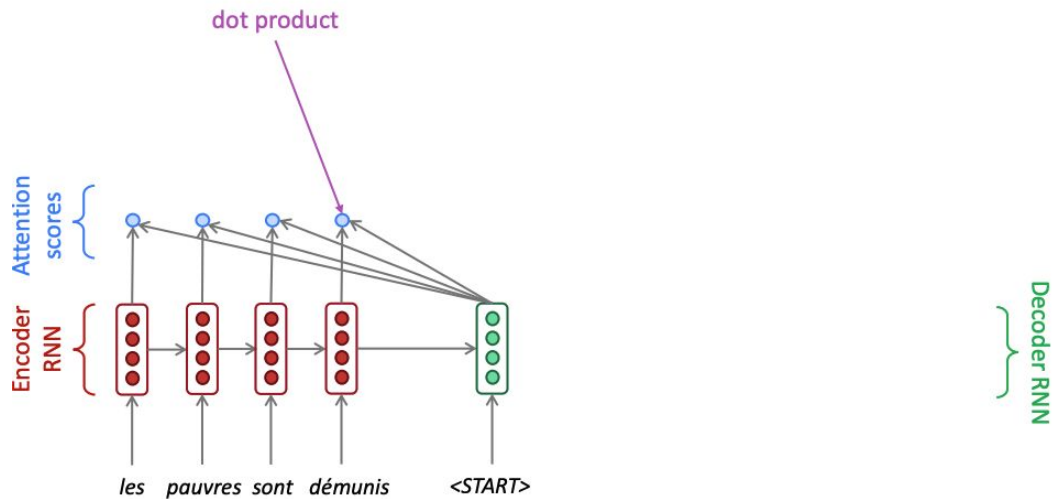
- 기계번역 예시
- 디코더 step 1 에서의 어텐션
with 인코더 2



1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

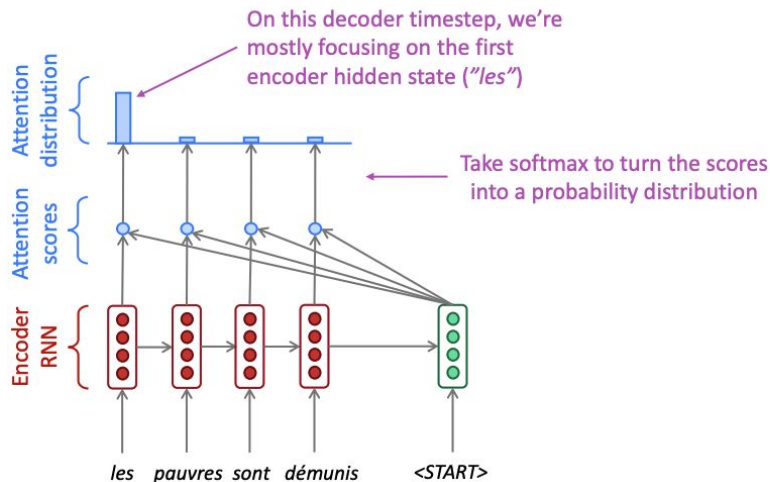
- 기계번역 예시
- 디코더 step 1 에서의 어텐션
with 인코더 4



1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

- 기계번역 예시
- 계산된 어텐션 스코어 기반으로
어텐션 분포(Attention distribution) 계산하기

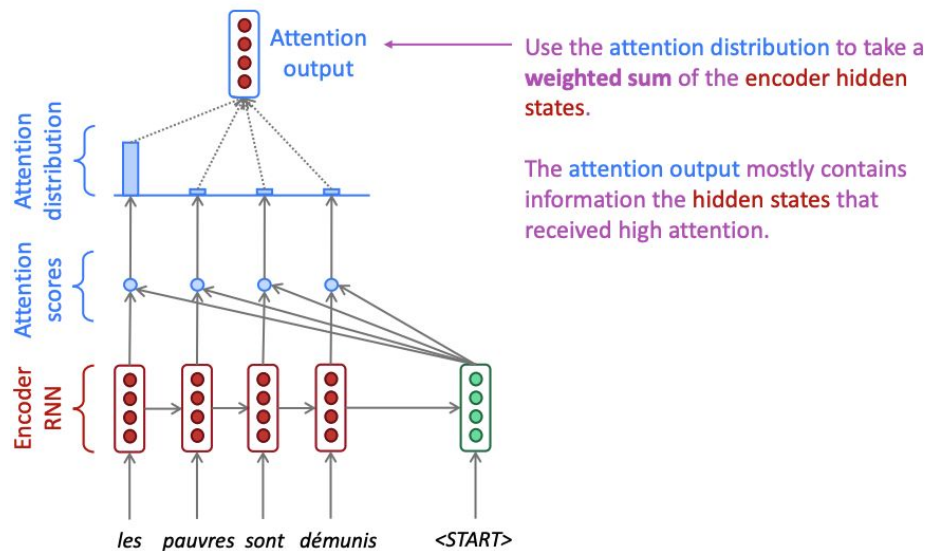


Decoder RNN

1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

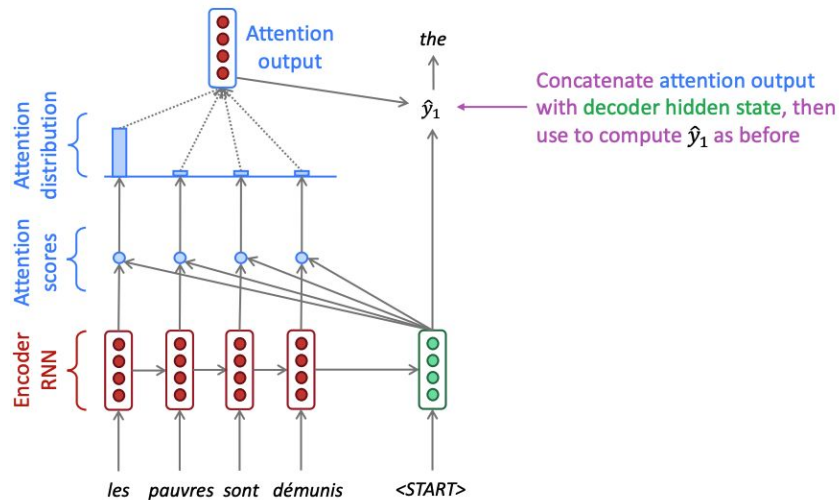
- 기계번역 예시
- 어텐션 분포(Attention distribution)를 활용하여 인코더 히든스테이트(hidden state)의 가중치 합(weighted sum)을 구한다.



1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

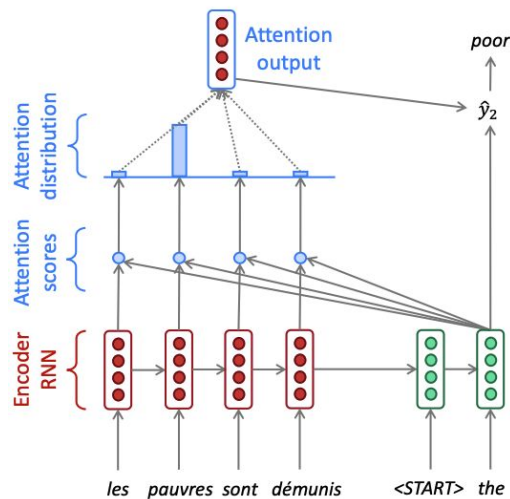
- 기계번역 예시
- 인코더 히든스테이트(hidden state)의 가중치 합(weighted sum) 을 디코더의 현재(t) 히든스테이트에 concat 해주고 $\hat{y}_t$ 으로 사용



1. 어텐션(Attention)이란?

1.2 RNN + Attention 구조

- 기계번역 예시
- 인코더 히든스테이트(hidden state)의 가중치 합(weighted sum)을 디코더의 현재(t) 히든스테이트에 concat 해주고 y_{hat} 으로 사용



1. 어텐션(Attention)이란?

1.3 2가지 Attention 구조

- Bahdanau Attention

- 어텐션 가중치 (Attention Score):

디코더의 이전 hidden state h_{t-1} 와 인코더의 hidden state h_s 를 기반으로 어텐션 스코어를 계산

$$e_t(s) = v_a^\top \tanh(W_a[h_{t-1}; \bar{h}_s])$$

- Attention 가중치: softmax 사용하여 가중치 정규화

$$\alpha_t(s) = \frac{\exp(e_t(s))}{\sum_{s'} \exp(e_t(s'))}$$

- 컨텍스트 벡터 계산 (Context Vector):

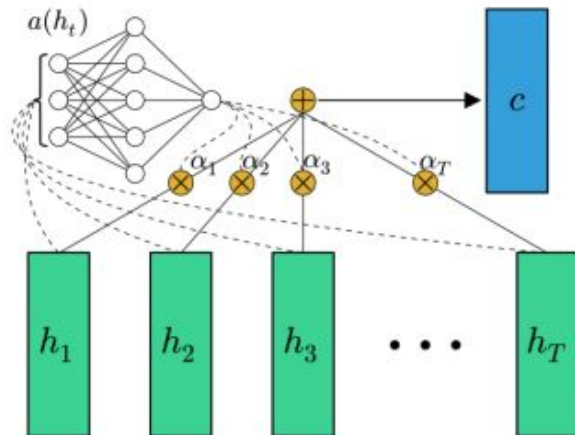
각 인코더 h 에 가중치 곱해서 컨텍스트 벡터 생성

$$c_t = \sum_s \alpha_t(s) \bar{h}_s$$

- 최종 출력 (Final Output):
concat 후 선형변환 후 활성화함수 적용

$$\hat{h}_t = \tanh(W_c[c_t; h_t])$$

Bahdanau attention



1. 어텐션(Attention)이란?

1.3 2가지 Attention 구조

- Luong Attention

- Attention Score 계산 (Alignment):

h_t : 디코더 (t)에서의 hidden state, h_s 는 인코더의 출력, W 는 가중치 행렬

$$score(h_t, \bar{h}_s) = h_t^\top W \bar{h}_s$$

- Attention 가중치: softmax 사용하여 가중치 정규화

$$\alpha_t(s) = \frac{\exp(score(h_t, \bar{h}_s))}{\sum_{s'} \exp(score(h_t, \bar{h}_{s'}))}$$

- 컨텍스트 벡터 계산 (Context Vector):

각 인코더 h 에 가중치 곱해서 컨텍스트 벡터 생성

$$c_t = \sum_s \alpha_t(s) \bar{h}_s$$

- 최종 출력 (Final Output):

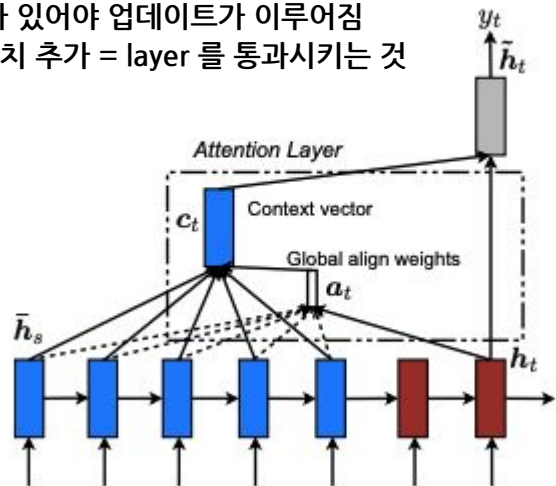
단순히 더해서 선형변환 후 활성화함수 적용

$$\hat{h}_t = \tanh(W_c[c_t; h_t])$$

학습 = 가중치 업데이트

즉, 가중치가 있어야 업데이트가 이루어짐

여기서 가중치 추가 = layer 를 통과시키는 것

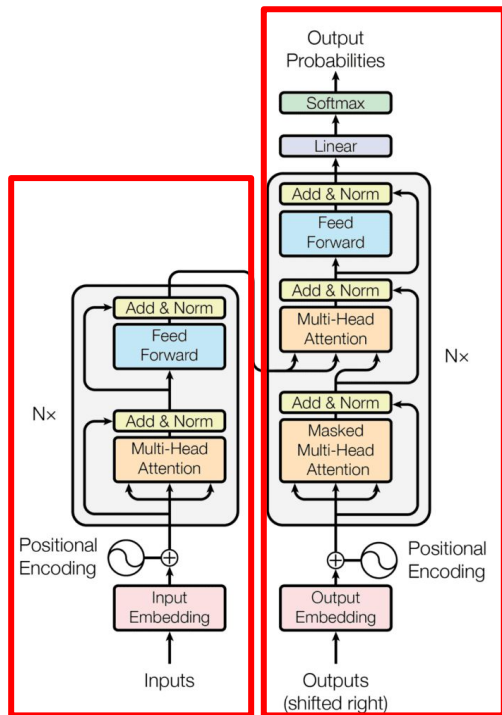


2.트랜스포머(Transformer)

2. 트랜스포머(Trnasformer)

2.1 Transformer

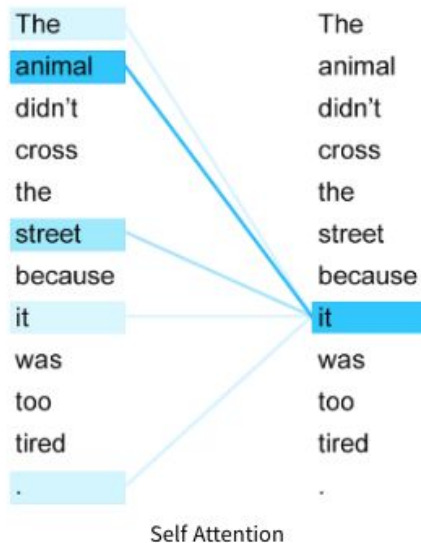
- “Attention is All you need” 논문에서 소개된 모델
- **기억해야할 포인트**
 - Seq2Seq Model
 - Multi-head Attention
 - Self-Attention
 - Positional Encoding



2. 트랜스포머(Trnasformer)

2.1 Transformer

- Self-Attention 이란?
 - 같은 문장 (Self) 내의 토큰들끼리 어텐션을 취하는 것
 - 기존 어텐션은 인코더-디코더간 어텐션이었던 반면,
인코더/디코더 각각에 입력된 문장 내에서 어텐션 진행

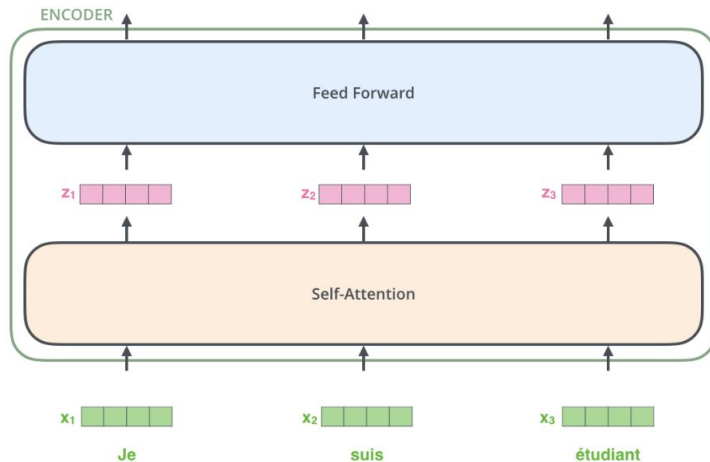
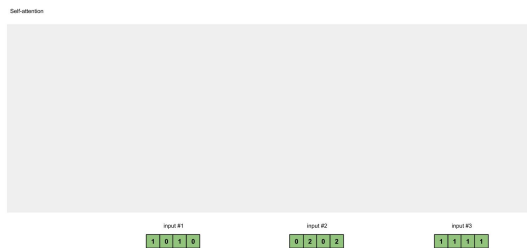


2. 트랜스포머(Trnasformer)

2.1 Transformer

- Self-Attention 에서 입력처리 방식

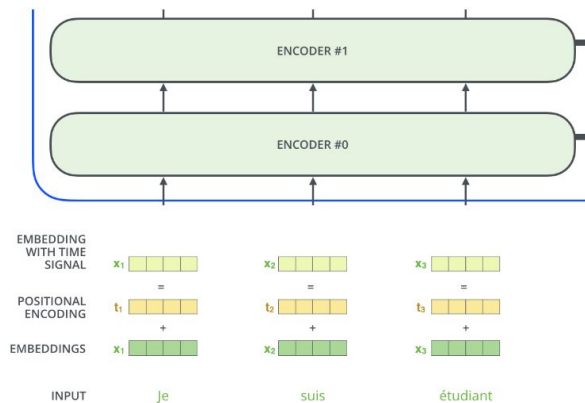
- 아래 과정을 통해서 Attention 이 동작하며, 기존 RNN + Attention 구조와 달리 **RNN 이 존재하지 않는다.**



2. 트랜스포머(Trnasformer)

2.1 Transformer

- Self-Attention 에서 입력처리 방식
 - 2가지 형태의 임베딩을 함께 전달한다.

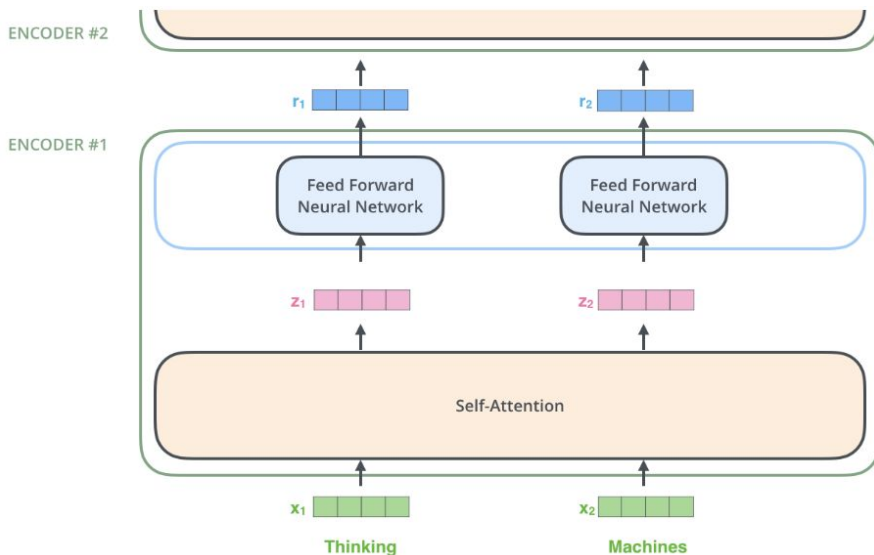
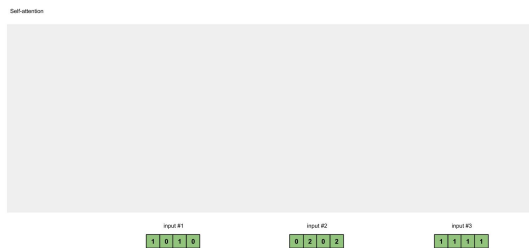


2. 트랜스포머(Trnasformer)

2.1 Transformer

- Self-Attention 에서 입력처리 방식

- 아래 과정을 통해서 Attention 이 동작하며, 기존 RNN + Attention 구조와 달리 **RNN 이 존재하지 않는다.**

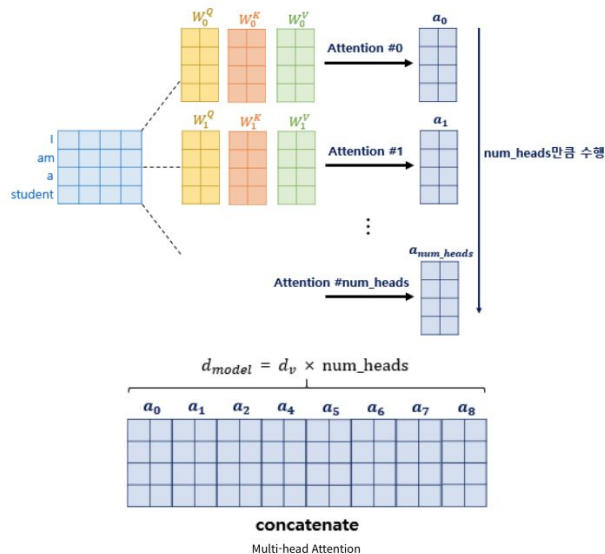


2. 트랜스포머(Trnasformer)

2.1 Transformer

- Multi-head Attention

- 입력 문장의 토큰 간의 **다양한 유형의 종속성**을 포착하기 위해, Attention (Query, Key, Value) 을 여러 개의 Head 로 나눠서 병렬로 계산
- 도출된 Attention Value 들은 **마지막에 concatenate** 를 통해 하나로 연결



2. 트랜스포머(Trnasformer)

2.1 Transformer

- Multi-head Attention

- 전체 hidden_size 를 num_attention_heads 로 나눠서 학습을 진행

```
self.num_attention_heads = config.num_attention_heads
self.attention_head_size = int(config.hidden_size / config.num_attention_heads)
self.all_head_size = self.num_attention_heads * self.attention_head_size
```

- 단, hidden_size 가 num_attention_heads 로 나눠떨어져야함

e.g) hidden_size = 1,024 & num_attention_heads = 8 → attention_head_size = 128

만약, num_attention_heads 가 7이면 에러 발생

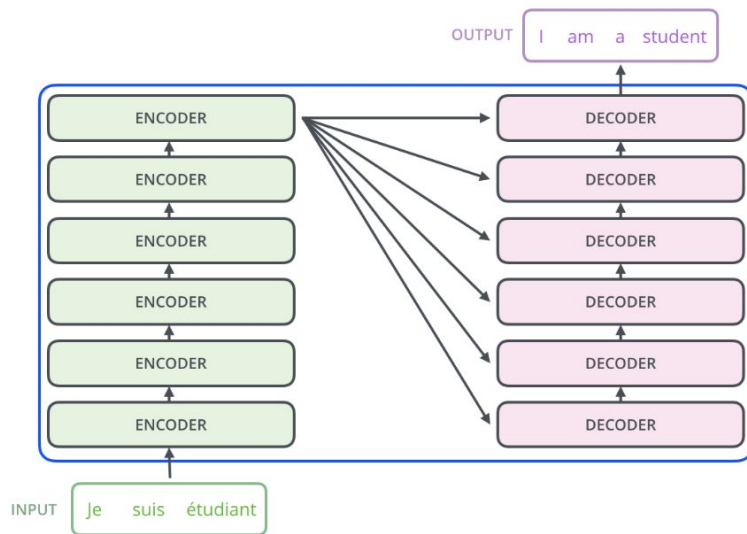
```
if config.hidden_size % config.num_attention_heads != 0 and not hasattr(config, "embedding_size"):
    raise ValueError(
        f"The hidden size ({config.hidden_size}) is not a multiple of the number of attention "
        f"heads ({config.num_attention_heads})"
    )
```

2. 트랜스포머(Trnasformer)

2.1 Transformer

- Encoder to Decoder

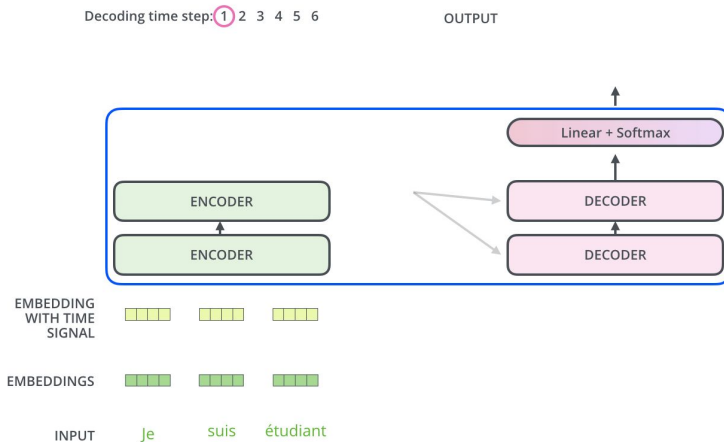
- Encoder 의 n 번(head 수 만큼) 연산 후 각 Decoder 에 전달한다.



2. 트랜스포머(Trnasformer)

2.1 Transformer

- Encoder – Decoder Attention(Cross Attention)
- Encoder 에서 어텐션을 통해 얻은 Q,K,V 에 대해서 **Decoder 에 K,V 를 전달함**

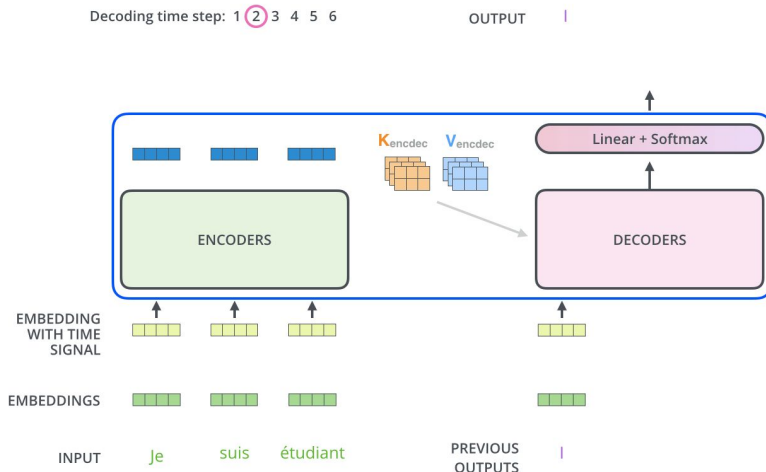


2. 트랜스포머(Trnasformer)

2.1 Transformer

- Encoder – Decoder Attention(Cross Attention)

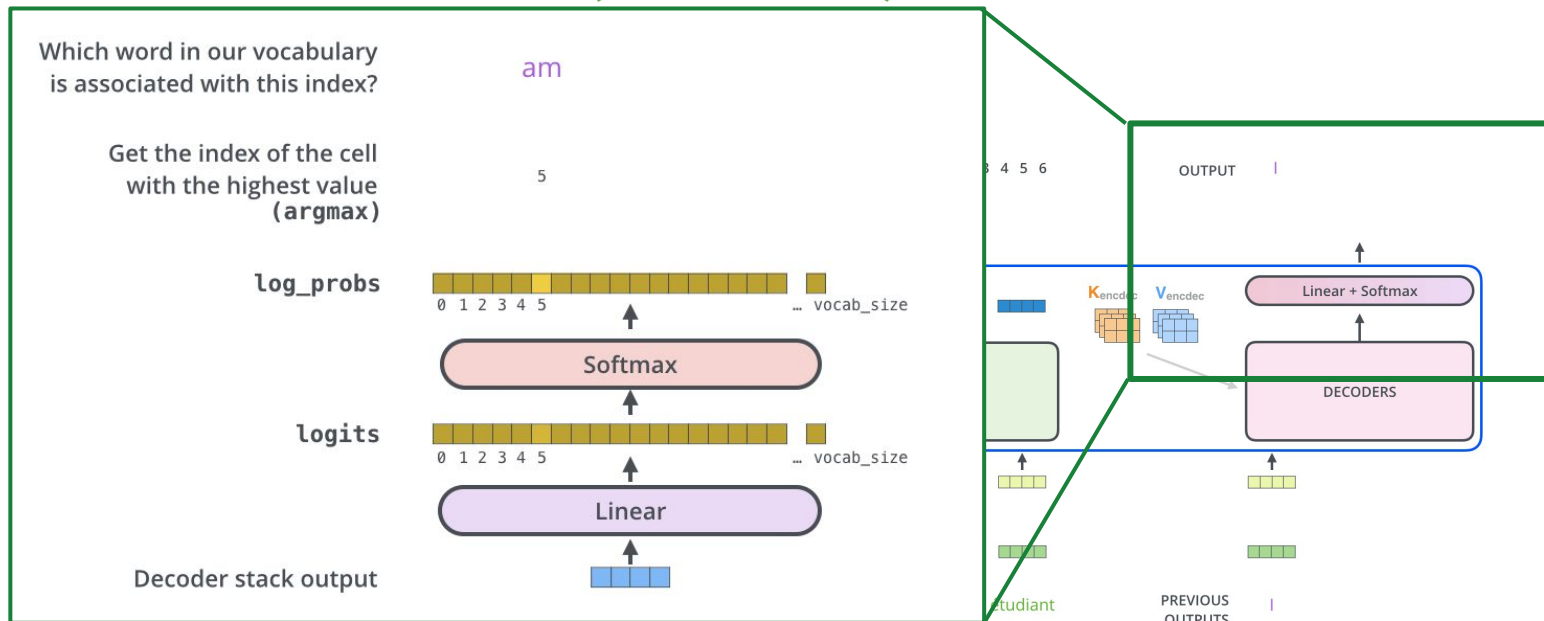
- Encoder 에서 어텐션을 통해 얻은 Q,K,V 에 대해서 **Decoder** 에 K,V 를 전달함
- 이후 Decoder 의 input 을 Q 로 활용하여 출력 생성



2. 트랜스포머(Trnasformer)

2.1 Transformer

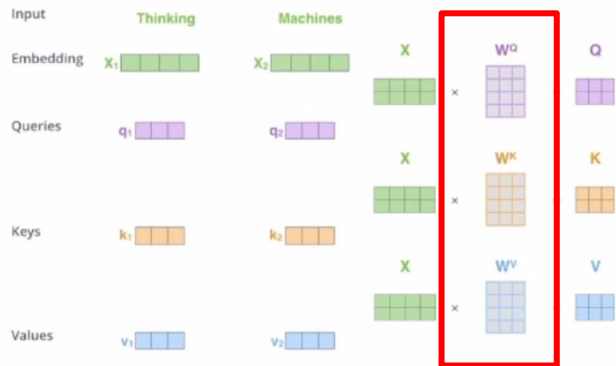
- Encoder – Decoder Attention(Cross Attention)



2. 트랜스포머(Trnasformer)

2.2 Query,Key,Value 연산

- 예시를 통해서 Query , Key , Value 간의 연산의 과정을 살펴봅니다.



1. 입력값 X 에 대해 가중치 (W_q , W_k , W_v)를 곱해서 Q, K, V 를 생성

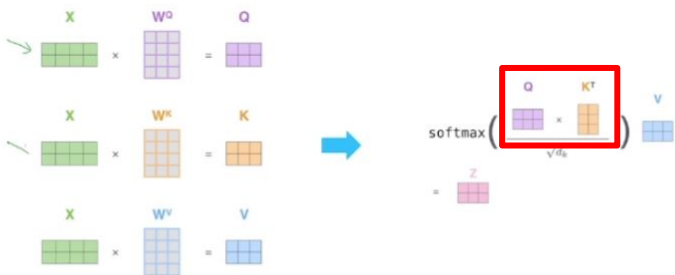
```
self.query = nn.Linear(config.hidden_size, self.all_head_size)
self.key = nn.Linear(config.hidden_size, self.all_head_size)
self.value = nn.Linear(config.hidden_size, self.all_head_size)
```

가중치를 곱해준다 = 학습이 가능한 상태를 만든다 = 모델을 통과시킨다.

2. 트랜스포머(Trnasformer)

2.2 Query,Key,Value 연산

- 예시를 통해서 Query , Key , Value 간의 연산의 과정을 살펴봅니다.



2. Query 에 대해서 각각의 Key 와 내적하여 attention scores 계산

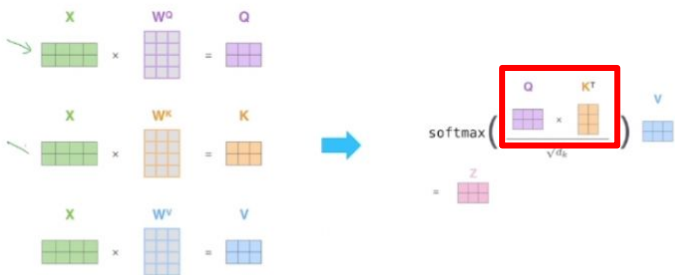
```
attention_scores = torch.matmul(query_layer, key_layer.transpose(-1, -2))
```

내적 위해 위치 변경
(2,3) >> (3,2)

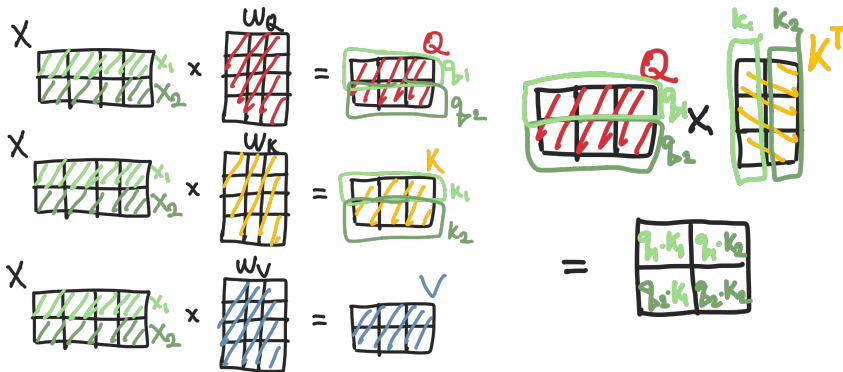
2. 트랜스포머(Trnasformer)

2.2 Query,Key,Value 연산

- 예시를 통해서 Query , Key , Value 간의 연산의 과정을 살펴봅니다.



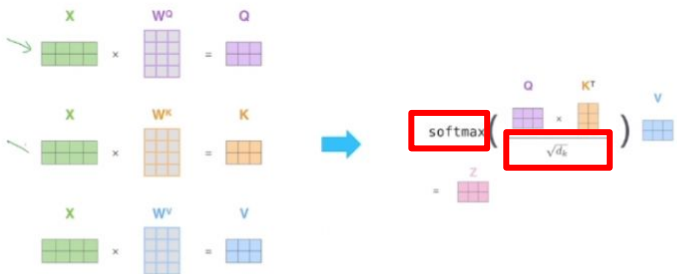
2. Query 에 대해서 각각의 Key 와 내적하여 attention scores 계산



2. 트랜스포머(Trnasformer)

2.2 Query,Key,Value 연산

- 예시를 통해서 Query , Key , Value 간의 연산의 과정을 살펴봅니다.



3. 내적인 값에 scaling 및 softmax 를 취해준다.

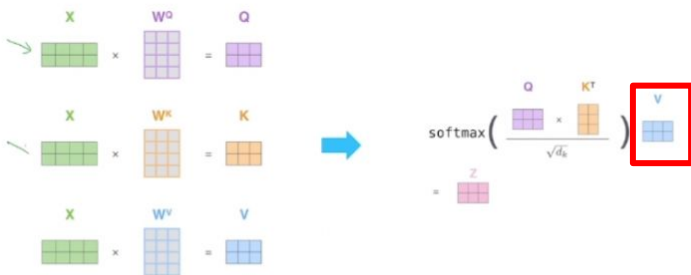
```
attention_scores = attention_scores / math.sqrt(self.attention_head_size)
```

```
attention_probs = nn.functional.softmax(attention_scores, dim=-1)
```

2. 트랜스포머(Trnasformer)

2.2 Query,Key,Value 연산

- 예시를 통해서 Query , Key , Value 간의 연산의 과정을 살펴봅니다.

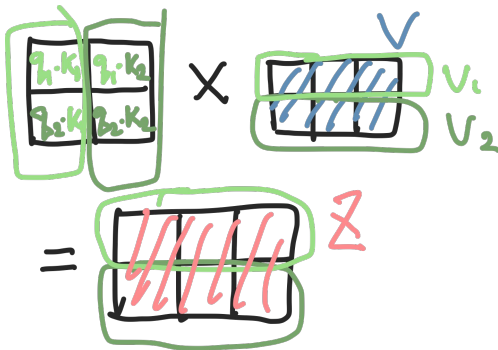


$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

The result Z is shown as a pink matrix. The Value matrix V is highlighted with a red box.

- softmax 까지 취해준 attention_probs 를 Value 에 곱해준다.

```
context_layer = torch.matmul(attention_probs, value_layer)
```



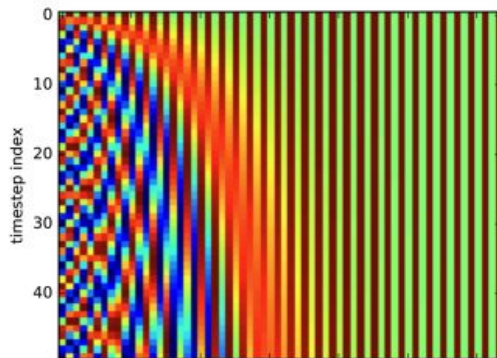
2. 트랜스포머(Trnasformer)

2.3 Positional Encoding

- RNN, CNN 과 달리, (Transformer는) 입력 시퀀스의 단어들이 어떤 순서로 들어왔는지 에 대한 정보가 누락되기에 이를 보완하기 위해 추가된 기능
- **sin,cos 과 같은 주기함수**를 통해 주기를 가지는 값을 제공

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



2. 트랜스포머(Trnasformer)

2.3 Positional Encoding

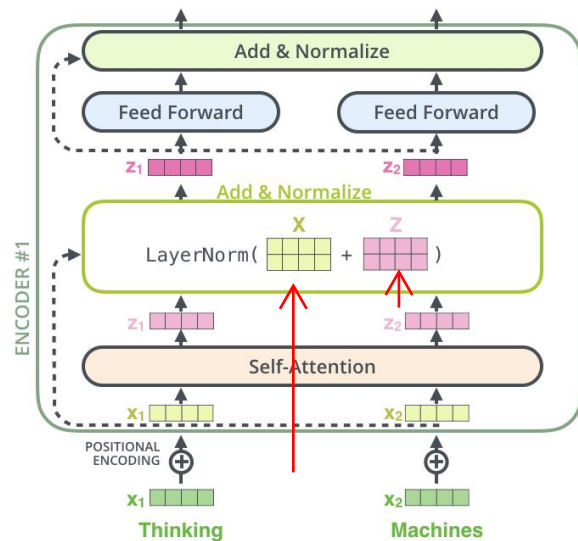
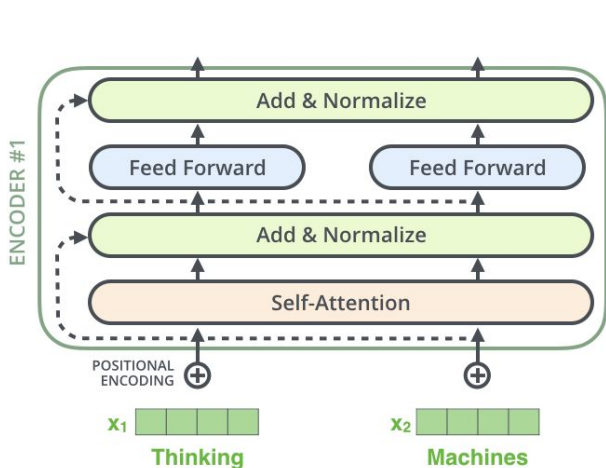
- RNN, CNN 과 달리, (Transformer는) 입력 시퀀스의 단어들이 어떤 순서로 들어왔는지 에 대한 정보가 누락되기에 이를 보완하기 위해 추가된 기능
- **sin,cos** 과 같은 **주기함수**를 통해 주기를 가지는 값을 제공
- **Input embedding 값에 더하여** 인코더와 디코더의 입력값으로 입력

```
embeddings = inputs_embeds + token_type_embeddings
if self.position_embedding_type == "absolute":
    position_embeddings = self.position_embeddings(position_ids)
    embeddings += position_embeddings
```

2. 트랜스포머(Trnasformer)

2.4 Residuals and LayerNorm

- 각 인코더의 각 하위 레이어(self-attention, ffnn)는 그 주변에 residual connection 이 있으며 레이어 정규화(layer - normalization) 도 뒤따른다.



3.허깅페이스 트랜스포머즈(transformers)

3. 허깅페이스 트랜스포머즈(trnasformers)

3.1 예시 테스트 - 낚시성 기사 분류하기

- 뉴스 기사의 제목과 본문을 입력으로 받은 후,
다음 6가지에 해당되는 경우를 분류하는 Task
- 데이터셋 : AI_HUB 낚시성 기사 탐지 데이터 활용
(전체 중 비낚시성 : 9,175개 , 낚시성 : 14,390개만 사용)

(11) 의문 유발형(부호)

(12) 의문 유바령(은닉)

(13) 선정표현 사용형

(14) 속어/줄임말 사용형

(15) 사실 과대 표현형

(16) 의도적 주어 왜곡형

3. 허깅페이스 트랜스포머즈(trnasformers)

3.1 예시 테스트 - 낚시성 기사 분류하기

뉴스 제목 : 도지코인도 5% 상승한 26센트에 거래돼 (상보)

낚시성 기사제목 _ 의문유발형(은닉)

뉴스 본문 : 최근 연일 하락했던 도지코인도 5% 반등에 성공했다.

도지코인은 27일 오전 6시10분 현재 (한국시간 기준) 코인마켓캡에서 24시간 전보다 5.05% 상승한 26.38센트를 기록하고 있다.

도지코인은 수시간 전 33센트까지 오르기도 했다.

도지코인이 상승한 것은 저가 매수세가 유입됐기 때문으로 보인다.

도지코인은 4월 20일 '도지데이'를 앞두고 43센트까지 오르는 등 랠리를 펼치다 도지데이 이후 5일 연속 급락했었다.

한편 같은 시각 한국의 거래사이트인 업비트에서 도지코인은 24시간 전보다 1.29% 상승한 313원에 거래되고 있다.

수정된 제목 : 최근 연일 하락했던 이것도 5% 반등에 성공

뉴스 제목: 폴스타, 브랜드 최초 SUV '폴스타3' 10월 세계 최초 공개\

뉴스 본문: 스웨덴의 고성능 프리미엄 전기차 브랜드 폴스타가 브랜드 최초의 스포츠유틸리티차량(SUV) '폴스타 3' 오는 10월 세계 최초로 공개한다. 8월 폴스타는 고성능 대형 전기 SUV 폴스타 3를 통해 자동차 폴스타3는 듀얼모터와 대형 배터리가 탑재될 예정이다.

국제표준배출가스시험방식(WLTP) 기준 600km 이상의 주행거리 인증을 목표로 하고 있다.

등급 최고 수준의 라이다 센서와 AI 분야 글로벌 선도기업 엔비디아의 중앙 집중식 컴퓨팅 기술 탑재를 통해 진보된 고속도로 자율 주행 기능을 선보일 예정이다.

폴스타3 는 올해 10월 새로운 영상과 이미지를 통해 세계 최초로 공개된다.

일부 시장에서는 공개 당일부터 폴스타3의 고객 주문이 시작된다.

폴스타 3의 생산은 2023년부터 시작된다.

미국과 중국에서 생산될 예정이다.

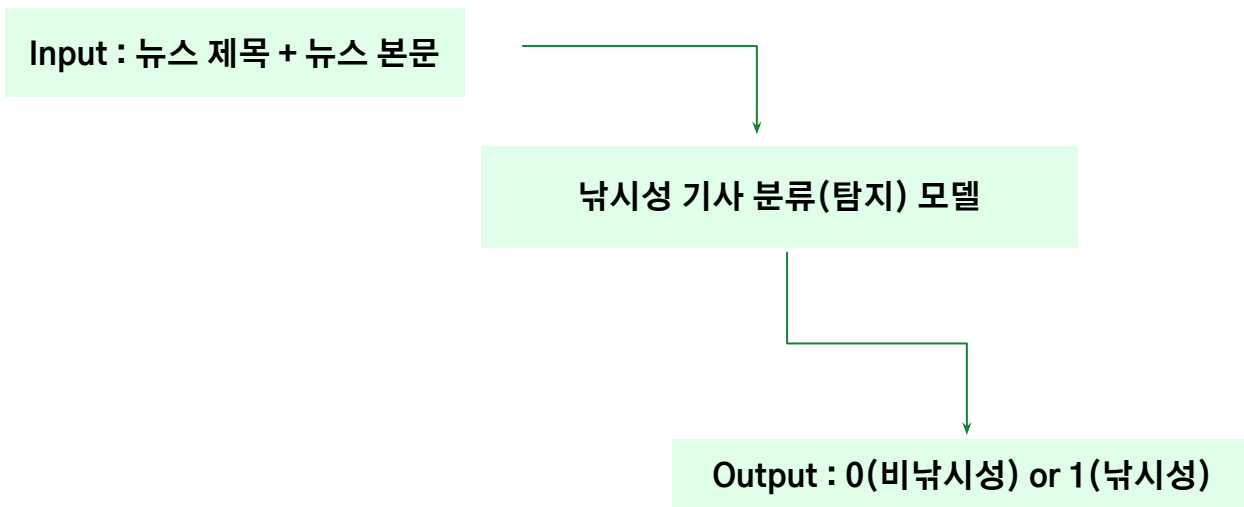
토마스 잉엔라트 폴스타 CEO는 "폴스타 3는 전통과 시대에 걸맞은 프리미엄 대형 전기 SUV\라며 \"폴스타의 디자인 정체성은 보다 강렬하고 개성 있게 진화했으며, '스포츠성'을 SUV에 반영해 폴스타의 핵심 가치인 폴스타는 자사 최초의 대형 전기 퍼포먼스 SUV 폴스타3를 시작으로 향후 3년간 매년 신차를 출시하고 2023년 말까지 30개 이상의 글로벌 시장에 진출할 예정이다.

2025년까지 29만대를 판매한다는 계획이다.

비낚시성 기사제목

3. 허깅페이스 트랜스포머즈(transformers)

3.1 예시 테스트 - 낚시성 기사 분류하기



3. 허깅페이스 트랜스포머즈(trnasformers)

3.1 예시 테스크 - 낚시성 기사 분류하기

- 코드 전체 구조

1. **환경설정** : 라이브러리 설치 및 데이터셋 로드(train.csv, test.csv)
2. **데이터셋 구축** : 입력 데이터(Input data)를 train/valid = 7.5:2.5로 나눠준(split)후
토큰나이징(Tokenizing)하여 torch dataset class 로 변환
3. **모델 및 토큰라이저 가져오기** : huggingface의 사전학습된(Pretrained) 모델 로드(load)
4. **모델 학습** : huggingface의 TrainingArguments & Trainer를 활용하여 학습데이터(train data)로 모델 학습
5. **추론 및 평가** : 학습된(Fine-tuned) 모델을 통해 평가 데이터(Test data) 추론 및 평가(Evaluation) 진행

3. 허깅페이스 트랜스포머즈(trnasformers)

3.2 Dataset & Tokenizing

1. 데이터 불러오기

1. 파일의 확장자에 맞게 pd.dataframe 형태로 로드
2. 예시) csv(comma-separated values) 파일 불러오기

```
pandas.read_csv(filepath or buffer, *, sep=_NoDefault.no_default,  
delimter=None, header='infer', names=_NoDefault.no_default,  
index_col=None, usecols=None, dtype=None, engine=None, converters=None,  
true_values=None, false_values=None, skipinitialspace=False, skiprows=None,  
skipfooter=0, nrows=None, na_values=None, keep_default_na=True,  
na_filter=True, verbose=False, skip_blank_lines=True, parse_dates=None,  
infer_datetime_format=_NoDefault.no_default, keep_date_col=False,  
date_parser=_NoDefault.no_default, date_format=None, dayfirst=False,  
cache_dates=True, iterator=False, chunksize=None, compression='infer',  
thousands=None, decimal='.', lineterminator=None, quotechar='\"', quoting=0,  
doublequote=True, escapechar=None, comment=None, encoding=None,  
encoding_errors='strict', dialect=None, on_bad_lines='error',  
delim_whitespace=False, low_memory=True, memory_map=False,  
float_precision=None, storage_options=None,  
dtype_backend=_NoDefault.no_default)
```

[source]

- filepath or buffer : 이곳에 csv data가 위치한 경로 입력
- sep or delimiter : 컬럼을 구분하는 구분자 지정. 기본값은 comma(,)
- header : 헤더가 있는 행 번호 지정. 기본적으로 첫번째 행 인식

3. 허깅페이스 트랜스포머즈(trnasmformers)

3.2 Dataset & Tokenizing

1. 데이터 불러오기

1. 파일의 확장자에 맞게 pd.dataframe 형태로 로드
2. 예시) csv(comma-separated values) 파일 불러오기

```
pandas.read_csv(filepath_or_buffer, *, sep=_NoDefault.no_default,
delimimter=None, header='infer', names=_NoDefault.no_default,
index_col=None, usecols=None, dtype=None, engine=None, converters=None,
true_values=None, false_values=None, skipinitialspace=False, skiprows=None,
skipfooter=0, nrows=None, na_values=None, keep_default_na=True,
na_filter=True, verbose=False, skip_blank_lines=True, parse_dates=None,
infer_datetime_format=_NoDefault.no_default, keep_date_col=False,
date_parser=_NoDefault.no_default, date_format=None, dayfirst=False,
cache_dates=True, iterator=False, chunksize=None, compression='infer',
thousands=None, decimal='.', lineterminator=None, quotechar='"', quoting=0,
doublequote=True, escapechar=None, comment=None, encoding=None,
encoding_errors='strict', dialect=None, on_bad_lines='error',
delim_whitespace=False, low_memory=True, memory_map=False,
float_precision=None, storage_options=None,
dtype_backend=_NoDefault.no_default)
```

[source]

- usecols : 데이터 읽어올때 사용할 컬럼들 리스트로 지정
- dtype : 각 컬럼의 데이터 타입 지정하는 딕셔너리
- encoding : 문자 인코딩 지정
e.g) 'utf-8' , 'cp949'

3. 허깅페이스 트랜스포머즈(trnasformers)

3.2 Dataset & Tokenizing

2. Pytorch Dataset class

데이터셋 클래스(Dataset class)는 우리가 가진 데이터셋을 **모델이 정의하고 있는 입력 형태로 가공**해주는 클래스이다.

데이터셋 클래스를 사용하는 이유는 **효율적인 데이터처리가 가능**하고, **확장성/호환성이 좋기** 때문이다.

- **다양한 데이터 소스와 형식에 대응**하는 DataLoader 구현이 손쉬움
- DataLoader와 함께사용하면 **배치 학습, 데이터 셔플링, 멀티 프로세싱**을 손쉽게 구현가능
- Pytorch의 **다른 구성요소들과 호환성**을 유지하면서 **구조화된 방식**으로 데이터 관리가능

3. 허깅페이스 트랜스포머즈(trnasformers)

3.2 Dataset & Tokenizing

2. Pytorch Dataset class

- `init`
초기 데이터 생성/전처리 방법 지정
- `len`
데이터셋의 전체 길이 반환
- `getitem`
`index` 에 해당되는 데이터 반환

```
import torch
from torch.utils.data import Dataset
class CustomDataset(Dataset):
    def __init__(self, text, labels):
        # 초기 데이터 생성 방법을 지정
        self.labels = labels
        self.data = text
    def __len__(self):
        # 데이터의 전체 길이 return
        return len(self.labels)
    def __getitem__(self, idx):
        # index 값을 주었을 때 반환되는 데이터의 형태 (x, y)
        label = self.labels[idx]
        text = self.data[idx]
        sample = {"Text": text, "Class": label}
        return sample
```

3. 허깅페이스 트랜스포머즈(transformers)

3.2 Dataset & Tokenizing

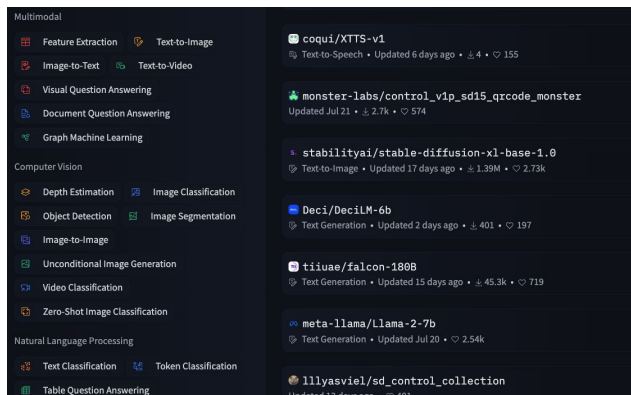
3. Tokenizing

- Huggingface 에서 사전학습된 토큰나이저 불러오기

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("model_checkpoint")
```

- 모델 체크포인트 찾기



3. 허깅페이스 트랜스포머즈(transformers)

3.3 Model & Trainer

1. Model

- Huggingface 에서 사전학습된 분류모델 불러오기

```
1 from transformers import AutoModelForSequenceClassification
2
3 model = AutoModelForSequenceClassification.from_pretrained("model_checkpoint")
```

- 사전 학습된 토큰라이저와 모델은 **동일한 체크포인트(checkpoint)여야 학습가능 함**
- 다른 체크포인트의 토큰라이저와 모델을 사용할 시, **호환성 문제 발생**

예시)

1. 모델A 는 “밥이 맛있다” 를 토큰라이저A 기준 [13, 2, 5] 로 알고 학습되어있다.

2. 그러나 토큰라이저B를 사용하면 “밥이 맛있다” 는 [42, 57, 21]로 토큰나이징 되어 엉뚱한 의미로 모델에 받아들여질 수 있다.

3. 허깅페이스 트랜스포머즈(transformers)

3.3 Model & Trainer

2. Trainer

(Huggingface) Trainer 는 모델을 학습, 평가, 최적화하기 위한 간편하고 확장 가능한 인터페이스를 제공하는 클래스이다.

(Huggingface) Trainer 를 사용하면 **배치 학습(Batch Learning)**, **학습 스케줄러(Learning Scheduler)** ,
학습 조기종료(EarlyStopping) 등의 기능들을 간단히 사용가능하다.

- **배치학습(Batch Learning)** : DataLoader 와 같이, batch size를 입력하여 batch learning이 가능함
- **학습 스케줄러(Learning Scheduler)** : 학습률(learning rate)을 특정 방식에 따라 조정하는 스케줄러 사용이 가능함
- **학습 조기종료(Early Stopping)** : 정해진 기준(threshold)만큼 학습이 진행되지 않는 횟수가 정해진 횟수(patience)만큼 연속하여 일어나면 학습을 조기 종료시킬 수 있다.

3. 허깅페이스 트랜스포머즈(trnasformers)

3.3 Model & Trainer

2. Trainer

- TrainingArguments 옵션 살펴보기

```
training_args = TrainingArguments(  
    output_dir=args_train.save_path + "results", # output directory  
    save_total_limit=args_train.save_limit, # number of total save model.  
    save_steps=args_train.save_step, # model saving step.  
    num_train_epochs=args_train.epochs, # total number of training epochs  
    learning_rate=args_train.lr, # learning_rate  
    per_device_train_batch_size=args_train.batch_size, # batch size per device during training  
    per_device_eval_batch_size=2, # batch size for evaluation  
    warmup_steps=args_train.warmup_steps, # number of warmup steps for learning rate scheduler  
    weight_decay=args_train.weight_decay, # strength of weight decay  
    logging_dir=args_train.save_path + "logs", # directory for storing logs  
    logging_steps=1, # log saving step.  
    evaluation_strategy="steps", # evaluation strategy to adopt during training  
    # `no`: No evaluation during training.  
    # `steps`: Evaluate every `eval_steps`.  
    # `epoch`: Evaluate every end of epoch.  
    eval_steps=1, # evaluation step.  
    load_best_model_at_end=True,  
)
```

- **save_total_limit** : 총 몇개의 ckt 를 저장할 것인지 지정하는 인자
- **warmup_steps** : **warmup** 을 몇 step까지 진행할 것인지 지정하는 인자
- **load_best_model_at_end** : 학습 종료 후 제일 성능 좋은 ckt를 저장하라는 인자

** warmup이란? 모델의 학습률을 점진적으로 증가시키는 과정

3. 허깅페이스 트랜스포머즈(trnasformers)

3.3 Model & Trainer

2. Trainer

- Trainer 옵션 살펴보기

```
trainer = Trainer(  
    model=model, # the instantiated 🤖 Transformers model to be trained  
    args=training_args, # training arguments, defined above  
    train_dataset=news_train_dataset, # training dataset  
    eval_dataset=news_train_dataset, # evaluation dataset  
    compute_metrics=compute_metrics, # define metrics function  
    callbacks=[MyCallback],  
    optimizers=(  
        optimizer,  
        get_cosine_with_hard_restarts_schedule_with_warmup(  
            optimizer,  
            num_warmup_steps=args_train.warmup_steps,  
            num_training_steps=len(news_train_dataset) * args_train.epochs,  
        ),  
    ),  
)
```

- compute_metrics**

사전에 정의해둔 metrics function 을 입력해주는 인자

```
def compute_metrics(pred):  
    """validation을 위한 metrics function"""  
    labels = pred.label_ids  
    preds = pred.predictions.argmax(-1)  
  
    # calculate accuracy using sklearn's function  
    acc = accuracy_score(labels, preds)  
  
    # calculate f1 score using sklearn's function  
    f1 = f1_score(labels, preds, average='micro')  
  
    return {  
        "accuracy": acc,  
        "f1": f1,  
    }
```

3. 허깅페이스 트랜스포머즈(trnasformers)

3.3 Model & Trainer

2. Trainer

- Trainer 옵션 살펴보기

```
trainer = Trainer(  
    model=model, # the instantiated 🤖 Transformers model to be trained  
    args=training_args, # training arguments, defined above  
    train_dataset=news_train_dataset, # training dataset  
    eval_dataset=news_train_dataset, # evaluation dataset  
    compute_metrics=compute_metrics, # define metrics function  
    callbacks=[MyCallback],  
    optimizers=(  
        optimizer,  
        get_cosine_with_hard_restarts_schedule_with_warmup(  
            optimizer,  
            num_warmup_steps=args_train.warmup_steps,  
            num_training_steps=len(news_train_dataset) * args_train.epochs,  
        ),  
    ),  
)
```

- callbacks

학습 조기종료(Early Stopping) 옵션을 설정해주는 인자

patience : 학습이 개선되지 않는 상황을 얼마나 참을지 결정하는 횟수

threshold : 학습이 개선되지 않음을 판단하는 임계점

```
MyCallback = EarlyStoppingCallback(  
    early_stopping_patience=3, early_stopping_threshold=0.001  
)
```

3. 허깅페이스 트랜스포머즈(transformers)

3.3 Model & Trainer

2. Trainer

- Trainer 옵션 살펴보기

```
trainer = Trainer(  
    model=model, # the instantiated 🤖 Transformers model to be trained  
    args=training_args, # training arguments, defined above  
    train_dataset=news_train_dataset, # training dataset  
    eval_dataset=news_train_dataset, # evaluation dataset  
    compute_metrics=compute_metrics, # define metrics function  
    callbacks=[MyCallback],  
    optimizers=(  
        optimizer,  
        get_cosine_with_hard_restarts_schedule_with_warmup(  
            optimizer,  
            num_warmup_steps=args_train.warmup_steps,  
            num_training_steps=len(news_train_dataset) * args_train.epochs  
        ),  
    ),  
)
```

`from transformers.optimization import get_cosine_with_hard_restarts_schedule_with_warmup`

- optimizers

학습최적화 알고리즘인 옵티마이저(optimizer) 및
학습 스케줄러(Learning Scheduler) 지정해주는 인자

3. 허깅페이스 트랜스포머즈(trnasformers)

3.4 Inference & Evaluation

1. Inference

- 학습된 모델의 체크포인트를 불러와 평가 데이터(test data)에 대한 추론(inference) 진행

```
def inference(model, tokenized_sent, device):  
    """학습된(trained) 모델을 통해 결과를 추론하는 function"""  
    dataloader = DataLoader(tokenized_sent, batch_size=args_infer.batch_size, shuffle=False)  
    model.eval()  
    output_pred = []  
    for i, data in enumerate(tqdm(dataloader)):  
        with torch.no_grad():  
            outputs = model(  
                input_ids=data["input_ids"].to(device),  
                attention_mask=data["attention_mask"].to(device),  
            )  
            logits = outputs[0]  
            logits = logits.detach().cpu().numpy()  
            result = np.argmax(logits, axis=-1)  
  
            output_pred.append(result)  
    return (np.concatenate(output_pred).tolist(),)
```

3. 허깅페이스 트랜스포머즈(trnasformers)

3.4 Inference & Evaluation

1. Inference

- `model.eval()`
 - 모델을 평가 모드로 전환하는 함수
 - Batch normalization , Dropout 과 같은 레이어들이 학습 모드와 다르게 작동
 - 학습을 하지 않는 추론이나 평가 시 사용
- `torch.no_grad()`
 - 자동 미분 기능 비활성화하는 함수
 - 모델 연산의 gradient를 저장하지 않는다.
 - 메모리 사용량을 줄이고 연산속도를 높일 수 있음

```
def inference(model, tokenized_sent, device):
    """학습된(trained) 모델을 통해 결과를 추론하는 function"""
    dataloader = DataLoader(tokenized_sent, batch_size=args_infer.batch_size, shuffle=False)
    model.eval()
    output_pred = []
    for i, data in enumerate(tqdm(dataloader)):
        with torch.no_grad():
            outputs = model(
                input_ids=data["input_ids"].to(device),
                attention_mask=data["attention_mask"].to(device),
            )
            logits = outputs[0]
            logits = logits.detach().cpu().numpy()
            result = np.argmax(logits, axis=-1)

            output_pred.append(result)
    return (np.concatenate(output_pred).tolist(),)
```

3. 허깅페이스 트랜스포머즈(transformers)

3.4 Inference & Evaluation

2. Evaluation

- 모델의 예측값(prediction)과 평가데이터(test data)의 라벨(label)값 사이에 평가 진행
 - sklearn.metrics 내 accuracy, f1_score 등 활용

```
def compute_metrics(pred):  
    """validation을 위한 metrics function"""  
    labels = pred.label_ids  
    preds = pred.predictions.argmax(-1)  
  
    # calculate accuracy using sklearn's function  
    acc = accuracy_score(labels, preds)  
  
    # calculate f1 score using sklearn's function  
    f1 = f1_score(labels, preds, average='micro')  
  
    return {  
        "accuracy": acc,  
        "f1": f1,  
    }
```

Thank You